

Smart Home EMS - Project Overview

1. config/ (System Configuration)

- config.yaml: Central configuration file. Stores safety limits (e.g., 2500W limit), ML parameters (ProtoNet distance thresholds), Reinforcement Learning settings (cooldown periods, comfort bounds), and MQTT settings.

2. scripts/ (Execution & Utilities)

- run_pipeline.py: The main orchestrator. It starts the 9-step processing pipeline (MQTT -> ML -> RL -> DB).
- start_broker.py: Launches a local amqtt broker.
- train_models.py: Offline training script for CNN, ProtoNet anchors, and Q-Table.
- generate_mock_ukdale.py: Generates synthetic energy dataset (mock_ukdale.h5).
- import_colab_weights.py: Utility to extract and load ML weights from Google Colab.

3. src/ (Core Backend & ML Pipeline)

- api/main.py: FastAPI REST endpoints and WebSocket bridge for the frontend.
- database/session.py: Asynchronous aiosqlite database logic with Write-Ahead Logging.
- hardware/mqtt.py: Async MQTT client manager (aiomqtt).
- models/protonet.py: CNN encoder and open-set classification logic.
- models/thermodynamics.py: ISO 7730 Fanger PMV thermal comfort Digital Twin.
- pipeline/safety.py: Real-time safety monitor (triggers emergency cutoffs).
- pipeline/watchdog.py: Soft anomaly detector using z-scores.
- pipeline/phantom_tracker.py: Tracks vampire loads when devices are off.
- rl/agent.py: Tabular Q-Learning agent that optimizes energy vs. comfort.

4. backend/ (Simulation & Models)

- scripts/simulate_esp32.py: Mocks 4 smart plugs generating 1Hz power readings.
- models/weights/: Directory where trained ML models (.pt, .json, .pkl) are saved.

5. frontend/ (React Dashboard)

A dark-mode Vite React application connecting to FastAPI WebSockets.

- App.jsx / index.css: Main layout and design system.
- components/DeviceCards.jsx: Live power state per device.
- components/RealTimeChart.jsx: Power traces and safety thresholds.
- components/DigitalTwin.jsx: Thermal comfort gauge and RL decisions.

6. notebooks/ & tests/

- notebooks/colab_train.py: Notebook for training on real UK-DALE data via GPUs.
- tests/test_pipeline.py: 45 integration tests verifying safety, ML, and API endpoints.

Smart Home EMS - Project Overview

7. Root Level Files

- Makefile: Command runner (make run, make test, make train_all).
- README.md / requirements.txt: Documentation and dependencies.